Web de Ropa — Plan de etapas y tareas por integrante

Nov 5, 2026 · @Salva

Resumen

En 10 días entregamos la **Segunda Parte del TP (integración sincrónica y asincrónica)** sobre el agregador de ropa: SOAP, REST con Swagger, RabbitMQ con productores y consumidores, dos servicios separados, una API externa real y un componente de IA simple. Todo lo demás queda para el Integrador.

Decisiones tomadas:

- **Sin compra:** se borra carrito, `Pedido`, `DetallePedido` y sus tests. Los eventos pasan a ser de catálogo, suscripción y recomendaciones.
- **Datos:** scraping de 1 o 2 tiendas reales; cada producto guarda la URL de la tienda oficial.
- **Stock:** solo "disponible / sin stock" y los talles disponibles.
- **Sin deduplicar:** la misma prenda puede aparecer una vez por tienda.
- **IA:** recomendaciones por contenido (un modelo simple), expuestas por REST y por cola.
- **Try-on, outfits y login con Firebase:** pasan al Integrador. No entran en estos 10 días.

Cómo leer este documento: cada integrante tiene su sección con tareas **por día**. Cada tarea dice qué hacer y cuándo está terminada. Si una tarea no está terminada al final de su día, se avisa ese mismo día en el grupo.

Qué pide la consigna y qué priorizamos

La Segunda Parte se aprueba con 7 requisitos; esos son la prioridad 1. El frontend no lo exige esta etapa: se hace lo mínimo para mostrar la demo.

Requisito (TP Segunda Parte)	Cómo lo cumplimos	Dueño	Prioridad	Evidencia para la entrega
Al menos 1 servicio SOAP con WSDL	Equivalencias de talles (AR, US, EU)	P2	1 — obligatorio	WSDL publicado + request y respuesta de prueba
Al menos 2 servicios REST con OpenAPI	Catálogo, suscripciones, recomendaciones y similares	P2 (Swagger) y cada dueño	1 — obligatorio	Swagger UI con todos los endpoints
Mensajería y colas	RabbitMQ con exchange, colas y cola de errores (DLQ)	P5	1 — obligatorio	Panel de RabbitMQ con mensajes

Requisito (TP Segunda Parte)	Cómo lo cumplimos	Dueño	Prioridad	Evidencia para la entrega
Productores y consumidores	`ProductoScrapeado`, `SuscripcionActivada`, `RecomendacionSolicitada`	P5 coordina; P2, P3, P4 implementan	1 — obligatorio	Logs de publicación y consumo
Arquitectura SOA / microservicios ligeros	2 servicios: backend y scraper, comunicados por la cola	P3 y P2	1 — obligatorio	`docker compose` con los servicios
Al menos 1 API externa real	**Mercado Pago** (pagos, ejemplo de la consigna)	P5	1 — obligatorio	Pago de prueba en sandbox que activa el premium
Componente de IA por REST o por cola	Recomendaciones por contenido	P4	1 — obligatorio	Tests con casos esperados + llamado REST y por cola
Segunda API externa	Ximilar: prendas similares	P3	2 — alta	Respuesta real guardada
Errores, logging, configuración externa	`@RestControllerAdvice`, logs, variables de entorno	P2	2 — alta	Error de ejemplo en Swagger
Frontend para la demo	Catálogo, detalle con "Ver en tienda", botón premium	P1	2 — alta	Capturas y video corto

Queda para el Integrador (no se toca en estos 10 días): try-on y maniquí, outfits, login con Firebase, historial de precios, búsqueda por imagen subida, más tiendas, pruebas de carga y los diagramas completos.

Plan de 10 días

El trabajo se ordena en 6 fases con 3 hitos de control. En cada hito el equipo se junta 30 minutos y prueba lo que hay en `main`; lo que no llegó se avisa y se replanifica ese mismo día.



plan de 10 días · 6 fases y 3 hitos

1. **Día 1 — Arranque.** GitHub configurado, `prototype` mergeado, modelo y eventos escritos, cuentas de Mercado Pago y Ximilar creadas.
2. **Días 2 y 3 — Base.** Se borra el código de compra y cada uno arma la base de su parte. **Hito 1:** catálogo con Swagger, RabbitMQ con la topología nueva, scraper con snapshot y recomendador con tests.
3. **Días 4 a 6 — Integraciones.** SOAP, scraper publicando en la cola, pago de prueba, recomendaciones por REST y por cola, similares. **Hito 2:** cada requisito obligatorio funciona por separado.
4. **Días 7 y 8 — Unir y probar.** Todo junto en `main` y en `docker compose`. **Hito 3:** desde acá no entran funciones nuevas, solo arreglos.
5. **Día 9 — Evidencias y documentación.**
6. **Día 10 — Ensayo y entrega.** Día de margen para imprevistos.

Cómo trabajamos en GitHub (explicado en detalle en el otro documento)

Usamos **GitHub Flow**: una sola rama principal (`main`) y una rama corta por tarea que entra por Pull Request. Es el esquema más simple para un equipo con poca experiencia; no usamos `develop`.

Reglas (se configuran el día 1)

1. `main` está protegida: nadie hace push directo. Todo entra por Pull Request con 1 aprobación y los tests en verde.
2. Una rama por tarea, con el número de rol y un nombre corto: `p3/scraper-tienda-1`, `p2/soap-talles`. Una rama vive 1 o 2 días como máximo.
3. Cada tarea de este documento es un **issue** con la etiqueta de su rol (`P1`…`P5`) y el milestone de su hito (Hito 1, 2 o 3).
4. Tablero en **GitHub Projects** con 4 columnas: Por hacer, En curso, En revisión, Hecho. Cada uno mueve sus propias tarjetas.
5. Commits chicos con prefijo: `feat:` (función nueva), `fix:` (arreglo), `test:`, `docs:`. Ejemplo: `feat: endpoint de filtros de catálogo`.
6. El Pull Request dice `Closes #<número de issue>` y completa la plantilla: compila, tests pasan, probado en local, Swagger actualizado si cambió la API.

7. Revisor fijo, que revisa en el mismo día: P1 ↔ P4, P2 revisa a P3, P3 revisa a P5, P5 revisa a P2. Se mergea con **Squash and merge** y se borra la rama.
8. Los conflictos los resuelve quien abrió el PR. Si no sabe cómo, pide ayuda en el grupo. Nunca `git push --force` sobre `main`.
9. Nunca se suben claves: van en un archivo `.env` (ignorado por git). En el repo solo queda `.env.example` con los nombres de las variables.
10. GitHub Actions corre `mvn test` y `npm test` en cada PR.

Comandos de todos los días

```
# Al empezar el día
git checkout main && git pull
git checkout -b p3/scraper-tienda-1   # rama nueva para la tarea

# Mientras trabajás
git add .
git commit -m "feat: normalizar talles de la tienda 1"
git push -u origin p3/scraper-tienda-1  # después, abrir el PR en GitHub

# Si main avanzó mientras trabajabas
git checkout main && git pull
git checkout p3/scraper-tienda-1 && git merge main
```

Rutina diaria (opcional)

- Daily de 15 minutos (o mensaje escrito en el grupo) siempre a la misma hora: qué terminé, qué hago hoy, qué me bloquea.
- Un bloqueo de más de 2 horas se avisa en el grupo en el momento, no al día siguiente.
- Al final de cada hito, 30 minutos juntos para probar todo integrado en `main`.

P1 — Frontend y evidencias

Qué te toca: el frontend mínimo para la demo y juntar las evidencias de la entrega. Revisás los PR de P4. Mientras P2 no publica los endpoints nuevos (día 3), trabajás con un JSON de ejemplo.

Día 1

- ☐ Levantar el frontend contra el backend de `main`. *Listo cuando:* ves el catálogo actual en el navegador.
- ☐ Borrar el carrito: `CarritoContext`, `CarritoMenu`, `useCarrito`, `crearPedido` y sus tests. *Listo cuando:* `npm run build` y `npm test` pasan.

Días 2 y 3

- ☐ Catálogo y detalle públicos: sacar `ProtectedRoute` de esas rutas. *Listo cuando:* se ven sin iniciar sesión.
- ☐ Adaptar el tipo `Producto` a los campos nuevos de P2 (`disponible`, `tallesDisponibles`, `urlOriginal`, `tienda`, `marca`).
- ☐ Filtros de categoría, tienda, género, precio mínimo y máximo y texto, enviados al backend como parámetros. *Listo cuando:* cambiar un filtro cambia los resultados.

- ☐ Detalle: "Disponible / Sin stock", talles y botón **Ver en tienda** que abre la URL oficial en otra pestaña.

Días 4 a 6

- ☐ Día 4: el login simple guarda el `usuarioId` y lo manda en el header `X-Usuario-Id` (lo acordás con P5).
- ☐ Día 5: bloque "Te puede gustar" en el detalle con `GET /api/recomendaciones?productoId=`.
- ☐ Día 6: página **Planes** con botón "Hacerme premium" que llama a `POST /api/suscripciones` y redirige a Mercado Pago; páginas `/pago/exito` y `/pago/error`.
- ☐ Día 6: bloque "Similares en otras tiendas" con `GET /api/productos/{id}/similares`. *Listo cuando:* muestra productos de otra tienda.

Días 7 y 8

- ☐ Probar el flujo completo con todos en `main` y arreglar lo que falle.
- ☐ Dockerfile del frontend. *Listo cuando:* `docker compose up` sirve la web.

Día 9

- ☐ Juntar las evidencias en `docs/evidencias/`: una captura por cada fila de la tabla de la consigna y un video de 2 a 3 minutos del flujo completo.

Día 10

- ☐ Ensayo de la demo con todo el equipo.

P2 — Backend core, catálogo y SOAP

Qué te toca: el backend central (modelo, catálogo, Swagger, SOAP) y el `docker-compose` final. Coordinás la integración en `main`. Revisás los PR de P3.

Día 1

- ☐ Mergear `prototype` a `main` y avisar al grupo. *Listo cuando:* `main` tiene RabbitMQ y compila.
- ☐ Escribir el modelo nuevo en `docs/modelo.md`: `Producto` (id, idExterno, nombre, descripción, precio, categoría, género, marca, tienda, `urlOriginal`, `imagenUrl`, `disponible`, `tallesDisponibles`), `Tienda` y `Usuario` (id, nombre, email, premium). *Listo cuando:* lo aprueban P3 y P5.

Día 2

- ☐ Borrar el código de compra: `Pedido`, `DetallePedido`, `PedidoController`, `PedidoCompraService`, `PedidoService`, `InventarioService`, `StockMapConverter`, `PUT /stock` y sus tests.
- ☐ Corregir `Categoria`: `REMPERA` pasa a `REMERA` y se agrega `VESTIDO`.
- ☐ Entidades, repositorios y DTOs del modelo nuevo; ids con `@GeneratedValue`; `tienda + idExterno` único.

Día 3

- ☐ `GET /api/productos` con filtros (categoría, tienda, género, precio mínimo y máximo, texto) y paginación; `GET /api/productos/{id}`; `GET /api/tiendas`.
- ☐ Seed nuevo con el snapshot de productos reales que te pasa P3.
- ☐ Swagger UI con springdoc (verificar la versión compatible con Spring Boot 4.1), `@RestControllerAdvice`, CORS global y credenciales en variables de entorno. *Listo cuando:* `/swagger-ui` muestra los endpoints. **Hito 1.**

Día 4

- ☐ SOAP de equivalencias de talles con Spring-WS: XSD, endpoint y tabla AR, US y EU para remeras, pantalones y calzado. *Listo cuando:* el WSDL se ve en el navegador y un request de prueba devuelve la equivalencia.

Día 5

- ☐ Consumidor de `ProductoScrapeado`: crea o actualiza el producto por `tienda + idExterno`. *Listo cuando:* correr el scraper llena la base.

Día 6

- ☐ Tests unitarios de `ProductoService`, del upsert y del SOAP. **Hito 2.**

Días 7 y 8

- ☐ Dockerfile del backend y `docker-compose` con Postgres, RabbitMQ, backend, scraper y frontend; `.env.example` con todas las variables.
- ☐ Probar todo integrado en `main`. *Listo cuando:* `docker compose up` levanta todo en una máquina limpia. **Hito 3: desde acá no entran funciones nuevas, solo arreglos.**

Día 9

- ☐ README: cómo levantar el proyecto, lista de endpoints, cómo probar el SOAP y un diagrama simple de arquitectura (servicios y cola).

Día 10

- ☐ Ensayo de la demo con todo el equipo.

P3 — Scraper y Ximilar

Qué te toca: el segundo servicio del sistema (el scraper, en la carpeta `scraper/`) y las prendas similares con Ximilar. Se recomienda hacer el scraper en Java con Spring Boot para que el resto pueda revisarlo. Revisás los PR de P5.

Día 1

- ☐ Elegir 2 tiendas de ropa argentinas. Priorizar las que publican su catálogo en JSON (muchas tiendas Shopify tienen `/products.json`; VTEX tiene endpoints públicos de catálogo). Verificarlo abriendo la URL en el navegador y revisar su `robots.txt`. *Listo cuando:* hay 2 tiendas con la URL del catálogo probada.

- ☐ Crear la cuenta de Ximilar (plan gratuito) y hacer un llamado de prueba con una foto de la tienda 1.

Día 2

- ☐ Crear el proyecto `scraper/`, leer el catálogo de la tienda 1 y convertir cada prenda al formato acordado con P2 (`docs/modelo.md`).
- ☐ Guardar el resultado en `scraper/snapshots/tienda1.json`. *Listo cuando:* tiene 30 prendas o más con imagen, precio, talles y URL oficial.

Día 3

- ☐ Pasarle el snapshot a P2 para el seed.
- ☐ Ordenar el código con una interfaz `ScraperTienda` (patrón **Strategy**) y la implementación de la tienda 1. **Hito 1.**

Día 4

- ☐ Publicar cada prenda como `ProductoScrapeado` en RabbitMQ, usando la configuración de P5. *Listo cuando:* los mensajes se ven en el panel de RabbitMQ.

Día 5

- ☐ Tienda 2 como otra implementación de `ScraperTienda`.
- ☐ Ejecución automática 1 vez por día y un comando para correrlo a mano en la demo.

Día 6

- ☐ `XimilarFacade` en el backend (patrón **Facade**): sube las imágenes del catálogo a una colección de Ximilar y busca similares. Con timeout y un perfil `mock` que devuelve respuestas fijas.
- ☐ `GET /api/productos/{id}/similares`. *Listo cuando:* devuelve prendas de la otra tienda. **Hito 2.**

Días 7 y 8

- ☐ Dockerfile del scraper.
- ☐ Tests del convertidor de cada tienda usando los JSON guardados (sin conexión a internet).

Día 9

- ☐ Documentar en el README: tiendas elegidas, cómo correr el scraper, créditos de Ximilar usados, y capturas de evidencia.

Día 10

- ☐ Ensayo de la demo con todo el equipo.

Cuidado con Ximilar: el plan gratuito da 1.000 créditos por mes. Subir una imagen a la colección cuesta 0,25 créditos, pero taggear una foto cuesta 15. En estos 10 días no se usa el tagging.

P4 — IA de recomendaciones y configuración de GitHub

Qué te toca: el componente de IA obligatorio (recomendaciones por contenido, por REST y por cola) y dejar armado GitHub el día 1. Revisás los PR de P1. El try-on no entra en estos 10 días.

Día 1

- ☐ Configurar GitHub: proteger `main` (1 aprobación y tests en verde), etiquetas `P1` a `P5`, milestones Hito 1, 2 y 3, tablero en GitHub Projects y plantilla de PR.
- ☐ GitHub Actions que corre `mvn test` y `npm test` en cada PR. *Listo cuando:* un PR de prueba muestra los tests corriendo.

Día 2

- ☐ Escribir la regla del recomendador en `docs/ia.md`: un puntaje por prenda según misma categoría o categoría complementaria, mismo género, precio parecido, misma marca y disponibilidad.
- ☐ Interfaz `EstrategiaRecomendacion` (patrón **Strategy**) y la clase `RecomendacionPorContenido`, probada con productos de ejemplo.

Día 3

- ☐ `RecomendadorFactory` (patrón **Factory**) que elige la estrategia.
- ☐ 5 tests unitarios con resultado esperado. **Hito 1.**

Día 4

- ☐ `GET /api/recomendaciones?productoId=&limite=` documentado en Swagger. *Listo cuando:* devuelve prendas parecidas al producto pedido.

Día 5

- ☐ Versión por cola: `POST /api/recomendaciones/solicitudes` responde 202 y publica `RecomendacionSolicitada`; un consumidor la procesa y guarda el resultado. *Listo cuando:* se ve el mensaje entrar y salir de la cola.

Día 6

- ☐ Segunda estrategia: `RecomendacionComplementaria` (arma un conjunto: parte de arriba, parte de abajo y calzado).
- ☐ Tabla de validación con 10 casos: producto de entrada, resultado esperado y resultado obtenido. **Hito 2.**

Días 7 y 8

- ☐ Probar las recomendaciones con P1 en el frontend y arreglar lo que falle.

Día 9

- ☐ Completar `docs/ia.md`: qué modelo se usa y por qué, cómo se valida, limitaciones, y capturas de evidencia del REST y de la cola.

Día 10

- ☐ Ensayo de la demo con todo el equipo.

Solo si todo lo anterior está terminado: probar una API de try-on con 3 fotos del catálogo, como preparación para el Integrador.

P5 — RabbitMQ y Mercado Pago

Qué te toca: la mensajería que usa todo el equipo y la API externa obligatoria (pago del plan premium con Mercado Pago). Revisás los PR de P2.

Día 1

- ☐ Escribir `docs/eventos.md`: para cada evento, quién lo publica, quién lo consume, en qué cola y el JSON de ejemplo. Eventos: `ProductoScrapeado`, `SuscripcionActivada`, `RecomendacionSolicitada`. *Listo cuando:* lo aprueban P2, P3 y P4.
- ☐ Crear la cuenta de desarrollador de Mercado Pago y los usuarios de prueba (sandbox).

Día 2

- ☐ Topología en RabbitMQ: un exchange para el sistema, una cola por consumidor y una cola de errores (DLQ) por cola.
- ☐ Reemplazar `PedidoCreado` por un publicador genérico que reutiliza las confirmaciones del broker que ya existen. *Listo cuando:* un test publica y consume un evento de prueba.

Día 3

- ☐ Dejar en `main` las clases de los eventos y la configuración, para que P3 y P4 publiquen sin repetir código.
- ☐ Identificación simple del usuario: el backend lee el header `X-Usuario-Id` (el login real con Firebase va al Integrador). **Hito 1.**

Día 4

- ☐ Entidad `Suscripcion` (usuario, estado, id del pago, fecha).
- ☐ `POST /api/suscripciones`: crea el pago del plan premium en el sandbox de Mercado Pago y devuelve la URL de pago. *Listo cuando:* la URL abre la pantalla de pago de prueba.

Día 5

- ☐ Recibir la confirmación: endpoint de webhook que consulta el estado del pago en Mercado Pago y, si está aprobado, publica `SuscripcionActivada`. Exponer el backend con ngrok para que Mercado Pago llegue.
- ☐ Plan B si el webhook se complica: al volver a `/pago/exito`, el backend consulta el estado del pago.

Día 6

- ☐ Consumidor de `SuscripcionActivada` que marca al usuario como premium (patrón **Observer**). *Listo cuando:* pagar en el sandbox deja `premium = true`. **Hito 2.**

Días 7 y 8

- ☐ Tests del publicador y de los consumidores; probar el flujo de pago con P1 en el frontend.

Día 9

- ☐ Documentar las colas y el flujo de pago; capturas del panel de RabbitMQ y del pago de prueba.

Día 10

- ☐ Ensayo de la demo con todo el equipo.

Dependencias: quién espera a quién

Si tu entrega aparece en esta tabla, alguien está esperándola: tiene que estar en `main` el día indicado. Mientras tanto, el que espera usa un JSON de ejemplo.

Día	Entrega	De	Para
1	`prototype` mergeado a `main`	P2	Todos
1	GitHub configurado (main protegida, tablero, Actions)	P4	Todos
1	`docs/modelo.md` (formato de producto)	P2	P1, P3, P4
1	`docs/eventos.md` (eventos y JSON)	P5	P2, P3, P4
3	Snapshot de productos reales	P3	P2
3	Clases de eventos y configuración de RabbitMQ	P5	P2, P3, P4
3	Endpoints de catálogo con Swagger	P2	P1, P4
4	`GET /api/recomendaciones`	P4	P1
4	`ProductoScrapeado` publicándose	P3	P2
4	`POST /api/suscripciones`	P5	P1
6	`GET /api/productos/{id}/similares`	P3	P1
8	`docker-compose` completo	P2	Todos

Riesgos y plan B

Cada riesgo tiene un plan B decidido de antemano, para no perder días discutiendo.

Riesgo	Plan B	Dueño
La tienda bloquea el scraper o cambia el sitio	Usar el snapshot guardado en `scraper/snapshots/` para la demo	P3

Riesgo	Plan B	Dueño
El webhook de Mercado Pago no llega a la máquina local	Consultar el estado del pago al volver a `/pago/exito`	P5
Ximilar no responde o se terminan los créditos	Perfil `mock` con respuestas fijas; Mercado Pago ya cubre la API externa obligatoria	P3
Alguien se atrasa más de 1 día	Se avisa en la daily y se pasa la tarea a quien esté adelantado; el frontend es lo primero que se recorta	Todos
La cátedra pide literalmente `PedidoCreado`	Consultar por mail el día 1	P2

Pendiente de decidir el día 1: qué persona toma cada rol (P1 a P5).